Technische Hochschule Deggendorf

MDM-M-2 Case Study Intelligent Systems in Production

Binary Classification of AOI Alarms into True Defects and False Calls

Second check of AOI results with supervised machine learning
(Siemens production data)

Bibliography Report

Team 1

Yu Han
Melisa Subasi
Meron Hagos
Alejandro Rivas

Under the supervision of:
Prof. Dr.-Ing. Hamidreza Heidari

Date of submission: 03.11.2026

Abstract

In electronics assembly, automated optical inspection (AOI) checks every solder joint after soldering and sends every flagged board to a manual inspection station. Most of these alarms are false calls: in public Siemens production data, about 99 % of the flagged solder joints are false calls. A second check with supervised machine learning after the AOI could release many false calls without manual inspection, and Siemens is working on this problem in its Erlangen electronics factory today.

This report reviews 18 sources, from the general to the specific. Earlier studies show that classifiers such as random forests can recognise false calls from AOI measurements. On the same Siemens data, however, models that meet the targets on a test set from the same period (at most 1 % missed defects, at least 40 % fewer manual checks) miss 5–15 % of the true defects on every later production period. The literature on dataset shift and concept drift, cases from manufacturing, and the authors of the data and of the study themselves point to drift, a change of the data over time, as the cause; an unstable method for setting the decision threshold adds to it. The standard remedy in the drift literature is to retrain the model regularly on recent data. In related applications, a binary fraud classifier with less than 1 % positives and a quality model on a semiconductor materials line, regular retraining reduced the loss in performance. For the second check of AOI results, this remedy has not been tested in the literature we reviewed.

Our project therefore reimplements the method of Pfab and Rothering with its own code, on the same data with the same models, and addresses the two causes they name: the threshold method and retraining. A design of experiments varies the threshold method (3 levels), the retraining schedule (6 levels), the retraining data (2 levels) and the model (2 levels). The deliverable is a simulation of the second check in production in which threshold and retraining are settings; its results are compared with the published result, and the targets of the original study (at most 1 % missed true defects, at least 40 % fewer manual checks) serve as reference lines.

Contents

Abstract i

1 Introduction 1

1.1 Motivation . . . 1

1.2 Key terms . . . 2

2 Materials and Methods 3

2.1 Reviewed literature . . . 3

2.1.1 Zero-defect manufacturing . . . 4

2.1.2 AOI in electronics assembly . . . 5

2.1.3 False calls of AOI . . . 5

2.1.4 Second check with supervised machine learning . . . 6

2.1.5 Classifiers for tabular data . . . 7

2.1.6 Evaluation with rare true defects . . . 7

2.1.7 Why models fail over time: drift and threshold . . . 7

2.1.8 Methods: threshold and retraining . . . 8

2.1.9 Bottom line: our project . . . 9

2.2 Summary of the reviewed works . . . 9

3 Discussion 10

3.1 Why the earlier results look better than they are . . . 10

3.2 Limitations of the reviewed sources . . . 10

4 Conclusion 11

References 11

List of Figures

1 The reviewed literature from general to specific. Each box is one level of the review, with its sources on the right; Section 2 follows the levels from top to bottom. The line below the funnel is our project. . . . 1

2 Missed true defects (horizontal) and false calls released (vertical) of the two models of [7], on the test set (stars, [7, p. 11, Table V]) and on the five later periods (dots, [7, p. 7, Table IV]). Further right is worse, higher is better. Green: the target of [7], at most 1 % missed and at least 40 % released. . . . 2

3 Share of flagged boards with a true defect per production week (bars, left axis; red: weeks above 3 %) and number of flagged boards per week (line, right axis). Data of [4], our analysis. 5

4 Missed true defects of the two models of [7] on the test set and on the five later periods (slices), mean and standard deviation of their runs [7, p. 7, Table IV; p. 11, Table V]. Dashed line: the target of at most 1 %. 6

5 Minimum number of true defects in the left-out data so that the threshold keeps the share of missed true defects below α with the stated confidence, $n \geq \log \delta / \log(1-\alpha)$ [14]; dark bars 95 %, light bars 90 % confidence; computed by us. 8

List of Tables

1 What a second check after AOI has reached so far. About 99 % of the flagged joints are false calls, so the share of false calls released is roughly the share of manual checks saved. 2

2 Key terms used in this report. 3

3 The 18 reviewed sources in the order of Figure 1: the finding we use and where it is in the source. 4

4 Studies on AOI false calls compared with our project. 9

5 Studies on drift and retraining compared with our project. 10

6 From the literature to the variables of our simulation (the two next steps named in [7]). . 10

1 Introduction

Zero-defect manufacturing relies on detecting defects before a product leaves the factory [1]. In electronics assembly, components are soldered onto printed circuit boards by surface-mount technology (SMT), and AOI is placed after soldering [2]. Every board that the AOI flags goes to a manual inspection station, where an operator decides whether it is a true defect or a false call [4]. A classifier after the AOI, a second check, could release the false calls without a manual check. This review covers 18 sources from 2001 to 2026 on this second check; image-based defect detection is outside its scope, because the second check starts after the AOI has made its measurements. Figure 1 shows the 18 sources from the general goal to our project; Section 2 follows it level by level.

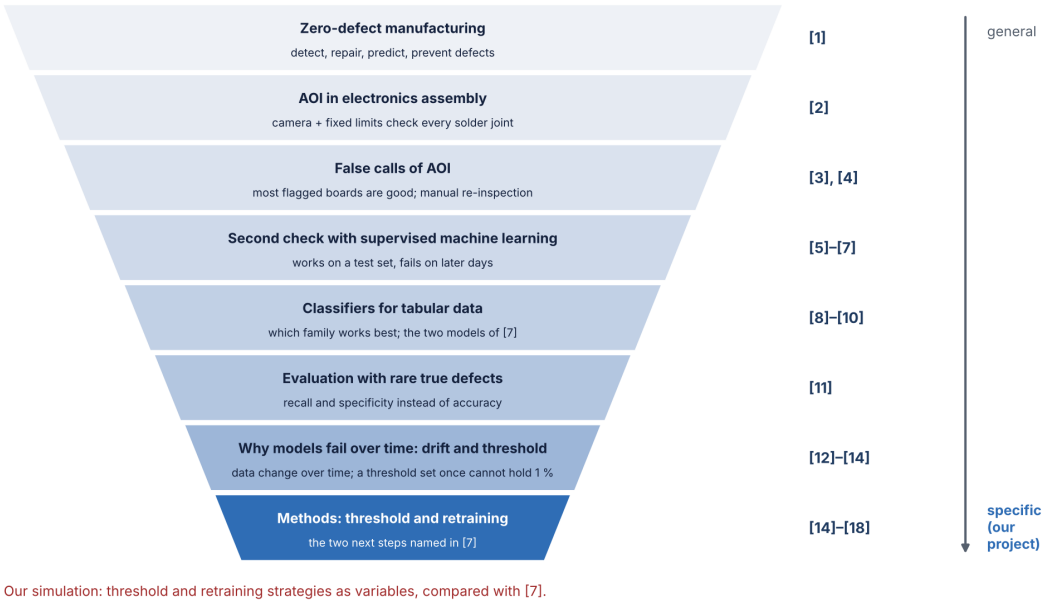


Figure 1: The reviewed literature from general to specific. Each box is one level of the review, with its sources on the right; Section 2 follows the levels from top to bottom. The line below the funnel is our project.

1.1 Motivation

Every board that the AOI flags is checked by a person, and about 99 % of the flagged solder joints are false calls [4, 7]. This costs operator time on every line, and it is a current problem: Siemens is working on an AI second check in its Erlangen factory today, with the target of 40 % fewer false calls [3]. Earlier studies show that a classifier on the AOI measurements can recognise many false calls [5, 6]. The only study that tests such a classifier on later production periods, however, finds that it fails there: it misses 5–15 % of the true defects instead of at most 1 % [7]. A factory cannot use a second check that lets defects through. The authors name two possible fixes, a better method for setting the threshold and handling drift, but test neither. Table 1 shows what has been reached so far and what is at stake.

Table 1: What a second check after AOI has reached so far. About 99 % of the flagged joints are false calls, so the share of false calls released is roughly the share of manual checks saved.

Source	Setting	False calls released (manual checks saved)	Missed true defects
Siemens [3]	industrial target, Erlangen	40 % (target)	–
Pfab & Rothering [7, p. 11, Table V]	test set, same period as training	79.7 % (BRF), 89.7 % (RF)	0.8 % (BRF), 1.0 % (RF)
Pfab & Rothering [7, p. 7, Table IV]	five later production periods	41.5–78.0 % (BRF), 39.7–76.0 % (RF)	5.3–9.4 % (BRF), 10.2–15.0 % (RF)
Our baseline	five later periods, 10 seeds, mean	57.9 % (BRF), 55.8 % (RF)	7.5 % (BRF), 9.1 % (RF)
Target of [7]	any period	≥ 40 %	≤ 1 %

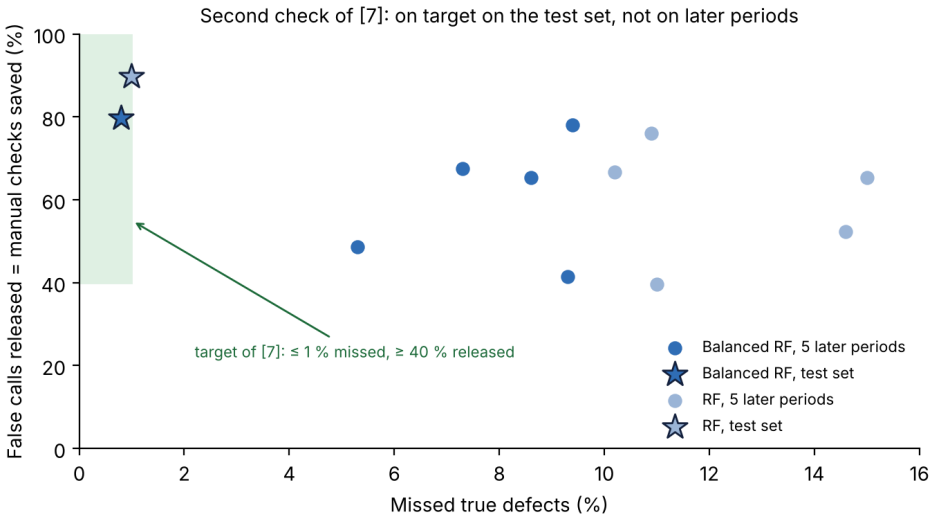


Figure 2: Missed true defects (horizontal) and false calls released (vertical) of the two models of [7], on the test set (stars, [7, p. 11, Table V]) and on the five later periods (dots, [7, p. 7, Table IV]). Further right is worse, higher is better. Green: the target of [7], at most 1 % missed and at least 40 % released.

On the test set, both models of [7] lie inside the target area (Figure 2): within the 1 % limit they release 80–90 % of the false calls, so 80–90 % of the manual checks would be saved. On all five later periods, both models still release about 40–78 % of the false calls, but they miss 5–15 % of the true defects, and all points lie to the right of the target area. Our baseline reproduces this pattern (Table 1). The benefit is therefore large but not yet usable. If the two fixes named in [7] move the later periods back into the target area, the manual inspection work on the line could fall by at least 40 %, the Siemens target [3]. Our motivation is to find out whether these two fixes make the second check usable in real production.

1.2 Key terms

Table 2 explains the terms used in this report. They follow the core study [7], so that our results can be compared with it.

Table 2: Key terms used in this report.

Term	Meaning
Automated optical inspection (AOI)	A camera system after soldering that measures every solder joint and compares each value with fixed limits [2].
Flagged joint, flagged board	A joint with at least one value outside its limits; a board with at least one flagged joint. Every flagged board goes to a manual inspection station [4].
True defect, false call	The label set at the manual station: the joint is really defective, or the alarm was false. True defect is the positive class.
Second check	A classifier between the AOI and the manual station that decides for each flagged joint whether it can skip the manual check.
Missed true defects (slip)	Share of true defects that the second check releases as false calls, $1 -$ recall. Target in [7]: at most 1 %.
False calls released (volume reduction)	Share of false calls that skip the manual station, the specificity. Target in [7]: at least 40 %.
Threshold	The score above which a joint is treated as a true defect; it is set so that recall reaches 99 % on known data.
Drift	A change of the data over time, so that a model trained on old data no longer fits new data [12].
Retraining	Training the model again on newer labelled data during production.

2 Materials and Methods

2.1 Reviewed literature

The review follows Figure 1 from top to bottom, one subsection per level. Table 3 lists the sources in the same order, with the finding we use and where it is in the source; page numbers refer to the version linked in the reference list.

Table 3: The 18 reviewed sources in the order of Figure 1: the finding we use and where it is in the source.

Level	Source	Type, year	Finding we use	Where in the source
1	[1]	journal review, 2020	280 articles (1987–2018); four strategies: detection, repair, prediction, prevention	abstract
2	[2]	journal review, 2020	how AOI works: cameras, lighting, feature extraction, fixed limits	abstract
3	[3]	press release, 2026	second check at Erlangen; targets 40 % fewer false calls, 85 % less engineering effort	web page
3	[4]	data article, 2024	132 days, one SMT line; labels set by people; "may contain distribution drifts of unknown type"	abstract; Limitations
4	[5]	conference paper, 2020	kNN, neural network, random forest after AOI; threshold for no extra slip	abstract; summarised in [7, p. 2]
4	[6]	conference paper, 2026	XGBoost best balance of recall and precision on Siemens data; LIME, SHAP	abstract
4	[7]	preprint, 2025	split in time order	p. 3, Sec. IV
			test set: 0.8 % (BRF) and 1.0 % (RF) missed; 79.7 % and 89.7 % false calls released	p. 11, Table V
			later periods: 5.3–9.4 % (BRF) and 10.2–15.0 % (RF) missed	p. 7, Table IV
			data clusters that "did just appear in a certain period"	p. 3, Fig. 2
			next steps: threshold and distribution drifts; "would fail from the first moment"	pp. 7–8, Sec. VII
5	[8]	conference paper, 2022	45 data sets: tree models remain state of the art on tabular data	p. 1, abstract
5	[9]	journal article, 2001	random forest: trees on random samples and features vote	abstract
5	[10]	technical report, 2004	balanced and weighted random forest for rare classes	pp. 2–3
6	[11]	journal review, 2009	accuracy misleading for rare classes; use recall, specificity	abstract
7	[12]	journal survey, 2014	drift = change of $p(X, y)$; retraining on a sliding window as baseline	definition and adaptation sections
7	[13]	journal article, 2025	controlled drift lowers all metrics of a weld-quality classifier	abstract
7, 8	[14]	journal article, 2018	empirical threshold does not hold an error bound; order statistic, $n \geq \log \delta / \log(1-\alpha)$	p. 2; p. 7 (arXiv copy)
8	[15]	journal article, 2002	re-estimate class shares from unlabelled data and adjust outputs	abstract
8	[16]	conference paper, 2004	drift detection: warning at 2, drift at 3 standard deviations	p. 4, Sec. 3
8	[17]	journal article, 2018	fraud, <1 % positives, 75 million transactions; daily update; recent data better	pp. 1–2; pp. 13–14
8	[18]	preprint, 2026	retraining every 5 vs. 100 batches: about 80 % fewer out-of-range predictions; frequency matters most	p. 9; p. 13

2.1.1 Zero-defect manufacturing

Level 1 of Figure 1, sources [1].

The funnel starts with the most general goal, zero-defect manufacturing: defects should be found before a product leaves the factory. Psarommatis et al. reviewed 280 articles on this topic published between 1987 and 2018 and grouped the approaches into four strategies: detection, repair, prediction and prevention of

defects [1]. Inspection after a production step belongs to the detection strategy, and a second check after the inspection is a way to make detection cheaper without letting more defects pass. This leads to the next level: how defects are detected in electronics assembly.

2.1.2 AOI in electronics assembly

Level 2 of Figure 1, sources [2].

In electronics assembly, defects are detected after soldering by automated optical inspection (AOI). Abu Ebayyeh and Mousavi describe how such systems work: cameras and lighting record each board, and the software pre-processes the images, extracts features and classifies each solder joint against fixed limits [2]. AOI replaced human visual inspection, which is slow and suffers from fatigue. For our project, this means that the input of a second check is already given: the AOI has measured each joint, and the second check works on these measurements, not on the images. AOI, however, raises many alarms that turn out to be false; this is the next level.

2.1.3 False calls of AOI

Level 3 of Figure 1, sources [3, 4].

Most AOI alarms are false calls. That this is a current industrial problem is shown by a press release of Siemens from October 2026 [3]. In its electronics factory in Erlangen, Siemens and NVIDIA prepare AOI programs on the computer before production, and when the AOI flags a possible defect, a second inspection step decides whether it is a real defect or a false call. The stated targets are up to 85 % less manual engineering effort, 40 % fewer false calls and more than 5 % higher first-pass yield. These are targets, not published results, but they show that the false-call problem is being worked on in industry today.

How large the problem is can be seen in data that Siemens published itself. Pfab et al. released 132 days of AOI data from one SMT line of Siemens AG in Germany [4]. Every row is a solder joint that the AOI flagged, with its AOI measurements and the label from the manual inspection station: true defect or false call. The authors point out two limits of the data: the share of wrong labels is unknown, because people set them, and "the data may contain distribution drifts of unknown type". This is the data set of our project, and its authors already name drift as one of its properties. Our own first analysis confirms it (Figure 3): the share of flagged boards with a true defect stays between 0.8 and 2.3 % for 15 weeks and rises to 3.2–6.3 % in the last four weeks, while the number of flagged boards in these weeks is not unusual. What changes is the mix of true defects and false calls, not the volume.

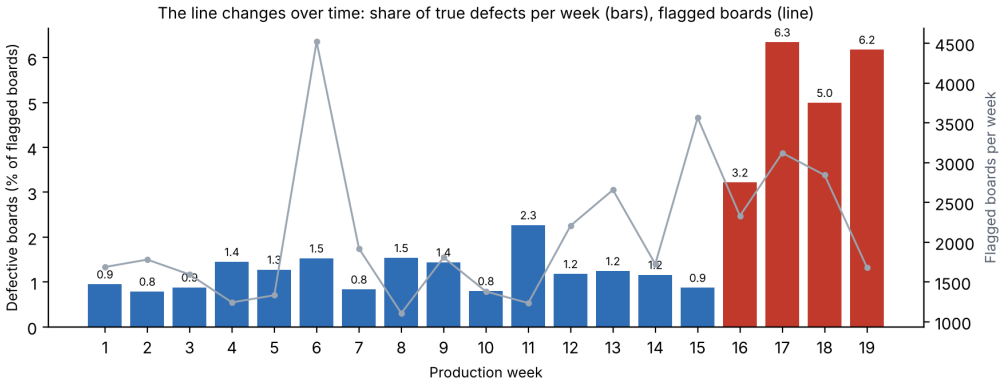


Figure 3: Share of flagged boards with a true defect per production week (bars, left axis; red: weeks above 3 %) and number of flagged boards per week (line, right axis). Data of [4], our analysis.

The obvious answer to so many false calls is a second check after the AOI, the next level.

2.1.4 Second check with supervised machine learning

Level 4 of Figure 1, sources [5–7].

A second check with supervised machine learning decides for each flagged joint whether it is a true defect or a false call. Whether such a step can recognise false calls from the AOI measurements was studied by Thielen et al. [5]. They compared k-nearest neighbours, a neural network and random forests and varied the size of the data set and the decision threshold, with the aim of recognising as many false calls as possible without more missed defects. k-nearest neighbours worked best on small data sets but became too slow on large ones, where neural networks and random forests performed better. Their data are not public, and whether the test data come after the training data in time is not reported. Albayari et al. later used AOI data from Siemens AG and compared XGBoost, random forest, a deep neural network and k-nearest neighbours with several train–test splits [6]. XGBoost gave the best balance of recall and precision, and the explanation methods LIME and SHAP showed which AOI measurements drive the predictions. Both studies therefore show that a second check can recognise false calls, but neither tests the classifier on production periods that come after the training data.

This is the gap that Pfab and Rothering close, and their study is the core paper of our project [7]. On the data of [4], they split the rows in time order: the first half is used for modelling, with a stratified random 80/20 split into a tuning set and a test set, and the second half is cut into five later periods. Their goal is at most 1 % missed true defects (slip) and at least 40 % fewer manual checks (volume reduction). They tune k-nearest neighbours, random forests, balanced random forests, XGBoost and an AutoML model with Bayesian optimisation and 5-fold cross-validation; the decision threshold is the mean of the thresholds found in the five folds, and each model is trained once. Two models, a random forest and a balanced random forest, meet the goal on the test set, but on all five later periods they miss 5–15 % of the true defects (Figure 4), already in the first one. The authors conclude that the model "would fail from the first moment in production". A principal component analysis shows data clusters that "did just appear in a certain period of time", and the authors state that "various distribution drifts" occur in the data. As possible next steps they name "improving the method for setting the threshold and handling the distribution drifts", without testing either.

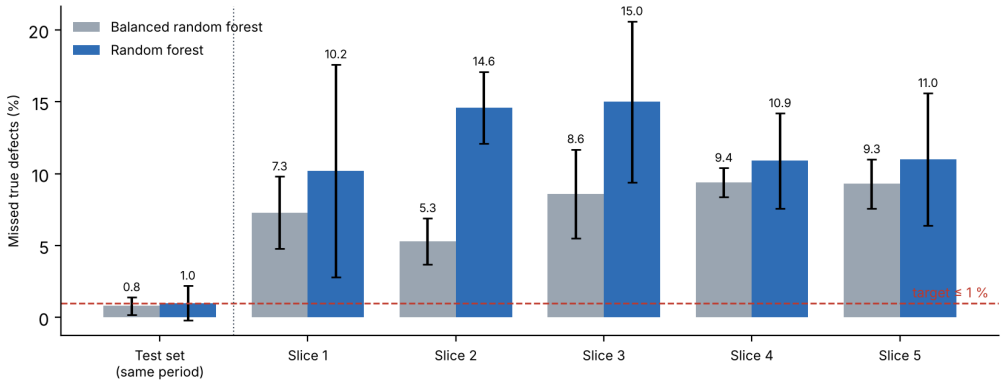


Figure 4: Missed true defects of the two models of [7] on the test set and on the five later periods (slices), mean and standard deviation of their runs [7, p. 7, Table IV; p. 11, Table V]. Dashed line: the target of at most 1 %.

Because the authors publish their code and run logs, their result can be checked directly. Two questions follow from this level: which classifiers to use, and how to evaluate them. They are the next two levels.

2.1.5 Classifiers for tabular data

Level 5 of Figure 1, sources [8–10].

AOI measurements are tabular data: each joint is a row with numerical values and a few categories. Which classifier family works best on such data was studied by Grinsztajn et al. on 45 tabular data sets [8]. Tree-based models such as XGBoost and random forests remained state of the art for medium-sized data, even without counting their speed advantage. This supports the tree ensembles used in [7] and means that deep tabular models are not needed to answer our question.

The two models of [7] that we reimplement go back to two original works. Breiman introduced the random forest, a set of decision trees that are each trained on a random sample of the data with a random subset of features at each split and then vote [9]; random forests reach an accuracy comparable to boosting, are robust to noise and give a measure of variable importance. Chen, Liaw and Breiman then proposed two variants for rare classes [10]: the weighted random forest, and the balanced random forest, which draws an equal number of cases from each class for every tree. The balanced variant fits our data, in which true defects are only about 1 % of the flagged joints. How these models should be evaluated when true defects are so rare is the next level.

2.1.6 Evaluation with rare true defects

Level 6 of Figure 1, sources [11].

With so few true defects, the choice of measure matters. He and Garcia explain why accuracy is misleading when one class is rare: a model that calls every joint a false call would be right 99 % of the time and still miss every defect [11]. They present class weighting and resampling as remedies and recommend class-wise measures such as recall and specificity. We therefore report, like [7], the missed true defects ($1 - \text{recall}$) and the false calls released (specificity), and not accuracy. With the tools and the measures fixed, the remaining question is why the second check of [7] fails on later periods.

2.1.7 Why models fail over time: drift and threshold

Level 7 of Figure 1, sources [12–14].

Pfab and Rothering name two causes for the failure on later periods: drift and the method for setting the threshold [7]. Drift is covered by the literature on concept drift. Gama et al. define concept drift as a change of the joint distribution $p(X, y)$ over time and group the ways of adapting to it into memory (which data to keep or forget), change detection and learning [12]. A model trained once on old data is therefore expected to fail when the data change.

That drift alone lowers the performance of a quality classifier in manufacturing was shown in a controlled experiment by Bauer et al. [13]. For the optical quality monitoring of friction stir welds (good or not good), they tested the same models on normal data and on data with controlled drift: other tool geometries, alloys and sheet thicknesses, more brightness and blur. All metrics dropped on the drifted data. Together with the warning in [4] and the data clusters in [7], this makes drift the most likely main cause of the failure over time.

The second cause is the threshold. Tong, Feng and Li show why the usual way of setting it is not enough [14]. When one type of error must stay below a bound α, the common practice of limiting the *empirical* error to α "does not satisfy the [...] error control objective", because the resulting classifiers "are still likely to have [...] errors much larger than α". A threshold set once on old data therefore cannot be trusted to keep the limit of at most 1 % missed true defects. Both causes have known remedies, which form the last level.

2.1.8 Methods: threshold and retraining

Level 8 of Figure 1, sources [14–18].

Threshold. For the first next step named in [7], "improving the method for setting the threshold", two methods exist. The first comes from the same work of Tong et al. [14]. Their umbrella algorithm works with any scoring classifier, including random forests: it takes the threshold as an order statistic of the scores of a left-out sample and so guarantees that the error stays below α with probability at least $1-\delta$. This needs a minimum size of the left-out sample, $n \geq \log\delta / \log(1-\alpha)$. For our bound of at most 1 % missed true defects, with $\alpha = 0.01$ and $\delta = 0.05$, the left-out data must contain at least 299 true defects; with fewer, no threshold can guarantee the bound (Figure 5). In the data of [4], true defects are about 1 % of the flagged joints, so 299 true defects need roughly 30,000 flagged joints. The cross-validation folds of [7] are large enough for this, but a short recent period may not be. This explains why a threshold for 1 % slip is unstable, and it gives a method, a threshold with a safety margin, to set it with a stated confidence.

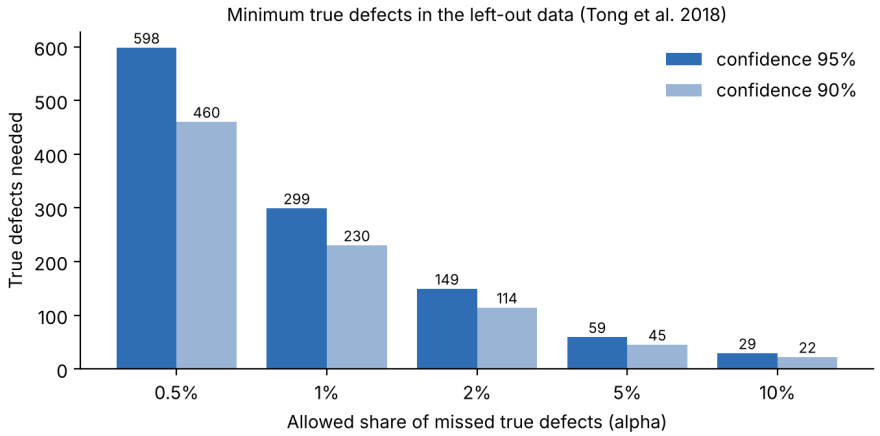


Figure 5: Minimum number of true defects in the left-out data so that the threshold keeps the share of missed true defects below α with the stated confidence, $n \geq \log\delta / \log(1-\alpha)$ [14]; dark bars 95 %, light bars 90 % confidence; computed by us.

The second method is needed when the share of true defects changes, as it does in the last weeks of the Siemens data (Figure 3). Saerens, Latinne and Decaestecker show that when the share of a class in use differs from its share in training, the outputs of a classifier no longer fit [15]. They re-estimate the new class shares from new, unlabelled data and adjust the outputs. For our project this means that a threshold fixed once cannot follow a changing share of true defects; it has to be set again for each period.

Retraining. For the second next step, "handling the distribution drifts within the dataset" [7], retraining is the basic remedy. In the survey of Gama et al., one learning mode is "retraining that discards the current model and builds a new model from scratch", and a sliding window that periodically retrains on the latest examples is "often used as a baseline" [12]. The open questions are when to retrain and whether it helps in practice. Gama et al. proposed the drift detection method (DDM), which retrains only when needed [16]. It tracks the online error rate of the model; a warning level is reached when the error plus its standard deviation exceeds the lowest value seen by two standard deviations, and a drift level at three. At a drift, a new model is learned from the examples since the warning. This is the alternative to retraining on a fixed schedule.

Evidence that regular retraining helps comes from two fields close to ours. In credit card fraud detection, Dal Pozzolo et al. studied a binary task with fewer than 1 % positives on a stream of more than 75 million transactions over three years [17]. The classifier is updated every day. A classifier trained on the most recent days ranks frauds better than one trained on data a week older, which the authors take as evidence

that the stream is non-stationary and that recent data are more informative; their strategy that uses the newest labelled transactions performs best throughout. This is the closest classification analogue to our task: binary, rare positives, a long daily stream, and regular retraining that helps.

In manufacturing, Gao et al. studied five years of data from a semiconductor materials line, where a random forest that predicts product quality degrades after a supplier change and with equipment wear [18]. Retraining every 100 batches gave "only marginal improvements", while retraining every 5 batches gave about 80 % fewer out-of-range predictions. The retraining frequency mattered more than the choice between a fixed and a growing training window, and new hyper-parameter tuning at each retraining was not worth its cost. The authors prefer a fixed schedule to drift-triggered retraining, which "will only be triggered after poor predictions have been observed". Their task is regression, not classification, and the study is a preprint, but it shows that in a real production setting the frequency of retraining is the setting that matters most.

2.1.9 Bottom line: our project

Bottom line of Figure 1.

Read from top to bottom, the funnel narrows the general goal of zero-defect manufacturing down to one open question. AOI detects defects but raises many false calls (levels 1–3). A second check with machine learning removes many of them on a test set, but fails on later production days (level 4). Tree models and class-wise measures are the right tools (levels 5 and 6). Drift and a threshold set once explain the failure (level 7), and methods for both exist and have worked in other fields (level 8). For the second check of AOI results, no study has tested them yet.

This is where our project starts. We reimplemented the method of [7] with our own code as the baseline. Reading their code, we found that the stored threshold belongs to the last model tried during tuning, not to the best model that is finally used; in our reimplementation the threshold is taken from the best model. On this baseline, the threshold method and retraining become the variables of a simulation, and every setting is compared with [7].

2.2 Summary of the reviewed works

Table 4 compares the studies on AOI false calls, and Table 5 compares the studies on drift and retraining. Table 6 maps the methods of level 8 to the variables of our simulation. Only [7] evaluates on later production periods, and no study on AOI false calls retrains its model (Table 4). Retraining helped in every study that tested it, but none of these studies is on AOI (Table 5).

Table 4: Studies on AOI false calls compared with our project.

	Data	Classifiers	Evaluation	Training
Thielen et al. [5]	not public	kNN, neural network, random forest	time order not stated	no retraining reported
Albayari et al. [6]	Siemens AG	XGBoost, random forest, DNN, kNN	several train–test splits	no retraining reported
Pfab & Rothering [7]	Siemens, public [4]	kNN, RF, balanced RF, XGBoost, AutoML	test set + 5 later periods	once
Our project	Siemens, public [4]	same as [7] (RF, balanced RF)	same as [7]	**once vs. retrained (6 schedules), 3 threshold methods**

Table 5: Studies on drift and retraining compared with our project.

	Domain	Task	Remedy studied	Main finding
Gama et al. [16]	data streams (benchmarks)	classification	retrain when the error rate rises (DDM)	drift can be detected from the error rate alone
Gama et al. [12]	survey	any	windows, forgetting, retraining	a periodically retrained sliding window is the usual baseline
Dal Pozzolo et al. [17]	credit card fraud	binary, <1 % positives	daily update on the newest labelled data	recent data rank frauds better
Bauer et al. [13]	weld quality (images)	binary	none (effect of drift)	all metrics drop under drift
Gao et al. [18]	semiconductor materials	regression	fixed schedule, window type	frequency matters most
Our project	SMT, AOI (Siemens)	binary, ≈ 1 % positives	threshold method and retraining schedule	tested in P3

Table 6: From the literature to the variables of our simulation (the two next steps named in [7]).

Variable	Option	What it does	Sources
Threshold	as in [7], corrected	mean of the five cross-validation thresholds, taken from the best model	[7]
	safety margin	1 % bound held with probability $1-\delta$; needs enough left-out true defects	[14]
	re-set per period	follows a changed share of true defects, also without labels	[15]
Retraining	never	train once, as in [7]	[7]
	on a fixed schedule	per period, weekly, every 3 days, daily; on all data so far or on a recent window	[12, 17, 18]
	on drift	retrain when the error rises	[16]

3 Discussion

3.1 Why the earlier results look better than they are

Most studies evaluate on a random or stratified split of data from the same period [5,6]. Under drift, such a split mixes past and future and hides the loss in performance [12]. The evaluation in time order of [7] is therefore the one that matches the use in a factory, and it is the reason why we build on this study. The measure matters as well: with about 1 % true defects, accuracy says little, and recall and specificity must be reported for the class that matters [11].

3.2 Limitations of the reviewed sources

Two of the central sources are preprints [7,18]; for [7], the published code and run logs allow us to check the results ourselves. The industrial targets of Siemens [3] are targets, not measured results. The manufacturing evidence for retraining [18] concerns regression, not classification, and the drift experiment of [13] works on images. The labels of the Siemens data come from operators and may contain errors [4].

Finally, all evidence for the AOI case comes from one production line; whether the findings transfer to other lines is not known.

4 Conclusion

The literature leads from the general goal of zero-defect manufacturing [1] to one open question. AOI raises many false calls that people check by hand [2, 4], and a second check with machine learning can release many of them [5, 6]. On later production days, however, it misses 5–15 % of the true defects instead of at most 1 % [7]. Drift [12, 13] and a threshold set once [14, 15] explain this failure, and retraining has helped in related fields [17, 18], but for AOI no study has tested either remedy.

Position of our project. In their discussion, Pfab and Rothering name as "potential next steps to stabilize the modeling" the "improving [of] the method for setting the threshold and handling the distribution drifts within the dataset" [7], without testing either. Our project focuses on exactly these two steps. We do not propose a new method; we test whether the standard remedies work for this real problem. Our reimplementation of [7], with the threshold taken from the best model, is the baseline. On it, a design of experiments varies the threshold method (as in [7] corrected, a safety margin [14], re-set per period [15]), the retraining schedule (never, per period, weekly, every 3 days, daily, on drift [16]), the retraining data (all data so far or a recent window [12]) and the model (random forest or balanced random forest), each setting with the ten seeds of [7]. The question is: how do missed true defects and released false calls on later production periods change when threshold and retraining change, compared with training once? Every result is shown next to the published result of [7]; its targets (at most 1 % missed true defects, at least 40 % fewer manual checks) serve as reference lines. In the end, the simulation becomes a small software tool in which threshold and retraining are settings. Whatever the result, it answers a question that the original study left open.

References

[1] F. Psarommatis, G. May, P.-A. Dreyfus, and D. Kiritsis, "Zero defect manufacturing: state-of-the-art review, shortcomings and future directions in research," *Int. J. Production Research*, vol. 58, no. 1, pp. 1–17, 2020, doi: 10.1080/00207543.2019.1605228.

[2] A. A. R. M. Abu Ebayyeh and A. Mousavi, "A review and analysis of automatic optical inspection and quality monitoring methods in electronics industry," *IEEE Access*, vol. 8, pp. 183192–183271, 2020, doi: 10.1109/ACCESS.2020.3029127.

[3] Siemens, "Siemens and NVIDIA advance AI-powered PCB inspection from Digital Twin to factory," press release, Plano, TX, USA, Oct. 21, 2026. [Online]. Available: https://news.siemens.com/en-us/siemens-nvidia-pcba-digital-twin-factory/

[4] K. Pfab, R. Eichler, A. Mallandur, and M. Rothering, "Data of automated optical inspection of surface-mounted technology electronic production," *Data in Brief*, vol. 53, Art. no. 110110, Apr. 2024, doi: 10.1016/j.dib.2024.110110. Data: Mendeley Data, v2, doi: 10.17632/99jzmh9658.2.

[5] N. Thielen, D. Werner, K. Schmidt, R. Seidel, A. Reinhardt, and J. Franke, "A machine learning based approach to detect false calls in SMT manufacturing," in *Proc. 43rd Int. Spring Seminar on Electronics Technology (ISSE)*, Demanovska Valley, Slovakia, 2020, pp. 1–6, doi: 10.1109/ISSE49702.2020.9121044.

[6] J. Albayari, P. Patra, D. Patel, and D. Santos, "Enhancing false call detection in automated optical inspection using XGBoost classifier and XAI algorithms," in *Flexible Automation and Intelligent Manufacturing (FAIM 2025)*, Lecture Notes in Mechanical Engineering. Cham, Switzerland: Springer, 2026, pp. 509–530, doi: 10.1007/978-3-032-05610-8_51.

[7] K. Pfab and M. Rothering, "Towards improved research methodologies for industrial AI: a case study of false call reduction," arXiv:2506.14521 [cs.LG], Jun. 2025. Source code and run logs: `https://gitlab.com/research9841542/FCR_1`. Open-access copy: PDF in our repository.

[8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022. Open-access copy: PDF in our repository.

[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[10] C. Chen, A. Liaw, and L. Breiman, "Using random forest to learn imbalanced data," Dept. of Statistics, University of California, Berkeley, Tech. Rep. 666, 2004. Open-access copy: PDF in our repository.

[11] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009, doi: 10.1109/TKDE.2008.239.

[12] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, Art. no. 44, pp. 1–37, 2014, doi: 10.1145/2523813.

[13] J. C. Bauer, S. Trattnig, F. Vieltorf, and R. Daub, "Handling data drift in deep learning-based quality monitoring: evaluating calibration methods using the example of friction stir welding," *J. Intelligent Manufacturing*, vol. 37, no. 2, pp. 759–774, 2025, doi: 10.1007/s10845-025-02569-6.

[14] X. Tong, Y. Feng, and J. J. Li, "Neyman-Pearson classification algorithms and NP receiver operating characteristics," *Science Advances*, vol. 4, no. 2, Art. no. eaao1659, 2018, doi: 10.1126/sciadv.aao1659. Open-access copy: PDF in our repository.

[15] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002, doi: 10.1162/089976602753284446.

[16] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence – SBIA 2004*, Lecture Notes in Computer Science, vol. 3171. Berlin, Germany: Springer, 2004, pp. 286–295, doi: 10.1007/978-3-540-28645-5_29. Open-access copy: PDF in our repository.

[17] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: a realistic modeling and a novel learning strategy," *IEEE Trans. Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, 2018, doi: 10.1109/TNNLS.2017.2736643. Open-access copy: PDF in our repository.

[18] M. Gao, J. M. Perathoner, A. L. Bonin, S. Eulig, and G. Klesse, "Robust and reliable AI for predictive quality in semiconductor materials manufacturing with MLOps and uncertainty quantification," arXiv:2605.07752 [cs.LG], 2026. Open-access copy: PDF in our repository.